Table of Contents

Chapter 1: Executive Summary & Architectural Context 3

1.1 Architectural Mandate & Key Metrics . . . 3

1.2 Scope Boundaries . . . 4

Chapter 2: Architectural Decisions (ADR-style) 4

2.1 ADR Summary Matrix . . . 4

2.2 ADR-01: Modular Monolith (vs Microservices / Serverless) . . . 5

2.3 ADR-02: Supabase Auth + Single Writer Role . . . 5

2.4 ADR-03: Provider Abstraction Layer for Gemini . . . 6

2.5 ADR-04: Project → PDF → Chunk → Embedding RAG Model . . . 6

2.6 ADR-05: Full Intervention Log (Audit Trail) . . . 7

2.7 ADR-06: Server-Sent Events (SSE) for Streaming . . . 7

2.8 ADR-07: pgvector in Supabase Postgres . . . 7

Chapter 3: User Journey Maps 8

3.1 Journey J1 — Onboarding & First Draft with Friction . . . 8

3.2 Journey J2 — Upload PDFs & Trigger Citation Sentinel . . . 9

3.3 Journey J3 — Writer's Block & Socratic Spacer . . . 10

Chapter 4: Use Case Diagram & Specification 11

4.1 Use Case Specification Table . . . 12

4.2 Include & Extend Relationships . . . 13

Chapter 5: System Architecture (Modular Monolith) 13

5.1 Module Responsibility Matrix . . . 14

5.2 Why Not Microservices for the MVP? . . . 15

5.3 Deployment Topology . . . 15

Chapter 6: Sequence & Data Flow Diagrams 15

6.1 Prompt Interceptor Sequence . . . 16

6.2 Citation Sentinel RAG Sequence . . . 16

6.3 Socratic Spacer Idle-Detection Sequence . . . 17

Chapter 7: Entity-Relationship Diagram 18

7.1 Entity Descriptions . . . 19
7.2 Cardinality Summary . . . 20
Chapter 8: Database Normalization (1NF → 2NF → 3NF) 21
8.1 Unnormalised Form (UNF) . . . 21
8.2 First Normal Form (1NF) . . . 22
8.3 Second Normal Form (2NF) . . . 22
8.4 Third Normal Form (3NF) . . . 22
8.5 Final 3NF Schema Summary (All 10 Tables) . . . 23
Chapter 9: AI Service Layer Design 24
9.1 Prompt Template Registry . . . 25
9.2 Token Budget & Rate Limiting . . . 25
9.3 Engine-Specific AI Calls . . . 26
Chapter 10: Security, Privacy & Non-Functional Considerations 26
10.1 Security Controls Matrix . . . 26
10.2 Latency Budgets . . . 27
10.3 Privacy & Academic Integrity . . . 27
Chapter 11: Conclusion & Migration Path 28
11.1 What Is In Scope for the MVP . . . 28
11.2 What Is Deferred . . . 28
11.3 Migration Path Beyond the Prototype . . . 28
11.4 Closing Argument . . . 28
Appendix A: Mermaid Diagram Source Code 29
A.1 Use Case Diagram (Figure 4.1) . . . 29
A.2 System Architecture (Figure 5.1) . . . 30
A.3 Interceptor Sequence (Figure 6.1) . . . 31
A.4 Citation Sentinel Sequence (Figure 6.2) . . . 32
A.5 Socratic Spacer Sequence (Figure 6.3) . . . 32
A.6 Entity-Relationship Diagram (Figure 7.1) . . . 33
A.7 AI Service Class Diagram (Figure 9.1) . . . 34

Chapter 1: Executive Summary & Architectural Context

Cogdoc is a web-based research-assisted writing IDE that deliberately refuses to generate final academic text. The Investigation Report (IR) and Project Specification Form (PSF) established that current generative-AI writing tools — ChatGPT, Gemini, Grammarly, Google Docs with Gemini — accelerate surface-level productivity but quietly erode the higher-order cognitive operations that academic writing is supposed to cultivate: synthesis, argument construction, evidence evaluation, and citation verification. The preliminary survey of 33 respondents confirmed the hypothesis: 69.6% of valid respondents admitted to copy-pasting AI-generated text into drafts without modification, only 26.3% strongly verified AI-generated citations against original sources, and 72.2% reported spending less time thinking about structure and flow when using AI. The architecture mandate that follows from these findings is therefore deliberately counter-intuitive: the system must be designed to introduce friction, not to remove it.

This document specifies the system architecture for the Cogdoc Minimum Viable Product (MVP). The architecture is engineered around four AI engines that interrupt, challenge, and verify the writer — never write for them. The **Prompt Interceptor** catches delegative prompts (“write a paragraph about X”) and reframes them as Socratic questions. The **Socratic Spacer** detects writer inactivity and surfaces a ghost question in the margin to provoke deeper thinking. The **Citation Sentinel** uses retrieval-augmented generation (RAG) over a user-uploaded PDF library to verify written claims and classify them as supported, partially supported, or unsupported. The **Living Margin** is the UI surface that renders every AI intervention beside the user’s text — never inside it — preserving the user’s authorship of the body content.

The architecture is constrained by three hard realities from the IR: a strict 60-day development schedule, a single developer (the student), and the academic prototype scope (not commercial SaaS). These constraints drive the choice of a **modular monolith** over microservices, **Supabase Auth** over a custom JWT implementation, and a **provider abstraction layer** over direct Gemini SDK calls. The chosen stack — React + Tiptap, FastAPI, Supabase PostgreSQL with pgvector, Gemini API, and Server-Sent Events (SSE) — is the same stack validated in the IR’s technical research chapter; this document specifies how those components fit together into a coherent, deployable, and gradeable system.

1.1 Architectural Mandate & Key Metrics

The following callout summarises the architectural constants that govern every design decision in the remainder of this document. Every trade-off must be evaluated against these constraints; any design choice that violates them requires explicit justification.

60	4	1	10	3
Development days available	AI engines (Interceptor, Socratic, Citation, Margin)	Actor role (Writer only)	Database tables at 3NF	Latency budgets (per engine)

Figure 1.1 — Architectural constants governing the Cogdoc MVP

1.2 Scope Boundaries

The scope in the IR (Section 1.5) carries directly into the architecture. The **in-scope** boundary includes the four AI engines, the custom Tiptap editor, the local RAG library, the prompt template registry with behaviour tags ([Provocation], [Lens: Perspective], [Structural Scaffold], [The Spark Gap]), and the technology stack listed above. The **out-of-scope** boundary excludes real-time collaborative editing, multi-author comments, version control, plagiarism detection, integration with institutional LMS systems, a mobile application, and advanced experimental features such as Devil's Autocomplete and the Blindspot Monitor. These exclusions are not architectural limitations; they are deliberate scope cuts that protect the 60-day timeline and keep the prototype gradeable.

> **Architectural North Star**
> The final text must always be written by the user. AI may interrupt, question, challenge, scaffold, and verify — but it must never produce the user's prose. Every architectural decision in this document is evaluated against this principle. Any feature that would allow the AI to write body content is, by definition, out of scope.

Chapter 2: Architectural Decisions (ADR-style)

This chapter records the seven most consequential architectural decisions for Cogdoc in the Architecture Decision Record (ADR) format. Each ADR follows the canonical structure of *Context*, *Decision*, and *Consequences* proposed by Michael Nygard. The intent is to make every trade-off auditable for the academic examiner and re-examinable when the prototype graduates from MVP to a research-grade tool. The decisions are not independent: ADR-01 (modular monolith) creates the boundary that ADR-03 (provider abstraction) and ADR-05 (full intervention log) live inside, and ADR-04 (RAG data model) shapes the schema that ADR-07 (pgvector) must store. Reading them in order therefore reflects how the architecture compounds.

2.1 ADR Summary Matrix

The table below lists every ADR with its decision in one line. Detailed rationale for each follows in Sections 2.2 through 2.8.

ID	Decision	Summary
ADR-01	Modular Monolith	Single FastAPI deployable with internal module boundaries; reject microservices for the MVP
ADR-02	Supabase Auth + 1 Role	Use Supabase built-in Auth (JWT); single Writer role; admin actions run via DB seeds, not UI
ADR-03	Provider Abstraction Layer	Define an AIService interface with a GeminiAdapter implementation; prompts versioned in DB
ADR-04	Project→PDF→Chunk→Embedding	Four-level RAG hierarchy with strict ownership; no cross-project PDF sharing in MVP
ADR-05	Full Intervention Log	Every AI intervention is persisted with prompt, response, target paragraph, user action, timestamp

ID	Decision	Summary
ADR-06	Server-Sent Events (SSE)	One-way streaming from server to client; reject WebSockets for the MVP (no bidirectional need)
ADR-07	pgvector in Supabase Postgres	Store embeddings in same DB as relational data; reject separate vector DB (Pinecone/Weaviate)

Table 2.1 — ADR summary matrix

2.2 ADR-01: Modular Monolith (vs Microservices / Serverless)

Context

Cogdoc must be built in 60 days by a single developer using RAD with Scrum-lite. The four AI engines (Interceptor, Socratic Spacer, Citation Sentinel, Living Margin) have different scaling profiles — the Citation Sentinel is compute-heavy (RAG + LLM call) while the Socratic Spacer is lightweight — but they share the same authentication, the same database, and the same Gemini API credentials. Microservices would require an API gateway, service discovery, inter-service auth, and distributed tracing, none of which a solo developer can operationalise in 60 days. Serverless functions would introduce cold-start latency that breaks the “instant margin” UX requirement.

Decision

Build a single FastAPI application organised into clearly bounded modules: auth, documents, pdf_manager, rag, ai_engines (with sub-modules for each engine), margin, ai_provider, and telemetry. Module boundaries are enforced by Python package structure and a dependency rule (modules may not import each other except through the ai_engines facade). Each module owns its own database tables and exposes a service interface, not raw DB access.

Consequences

Positive: single deploy command, single log stream, simpler local dev, no network hop between modules. Negative: a bug in any module can take down the whole app. Mitigation: module-level integration tests + graceful degradation (if ai_engines fails, the editor still loads). Migration path: each module is service-seam-ready — the ai_engines module can be extracted into a citation-service microservice if RAG load demands horizontal scaling.

2.3 ADR-02: Supabase Auth + Single Writer Role

Context

The IR (Section 2.3.7) requires basic authentication, secure file handling, and user-level data isolation. The PSF lists no admin-side concerns beyond seeding the prompt tag library. A custom JWT implementation would consume 5–7 development days for password hashing, refresh tokens, and rate limiting — time better spent on the AI engines.

Decision

Use Supabase built-in Auth (email/password and optional OAuth). Model exactly one actor: the **Writer** (a student or researcher). Admin operations — seeding prompt templates, managing behaviour tags, viewing aggregate telemetry — run through SQL migrations and a Django-admin-style script, not through a UI surface.

Row-Level Security (RLS) policies on every table enforce user_id-scoped access at the database layer, not in application code.

Consequences

Positive: zero auth code, battle-tested JWT issuance, RLS as a second line of defence. Negative: vendor lock-in to Supabase; no admin UI for non-technical operators. Acceptable for a prototype; the abstraction layer in ADR-03 contains the rest of the system so Supabase can be swapped later.

2.4 ADR-03: Provider Abstraction Layer for Gemini

Context

The IR selects the Gemini API for classification, embedding, and generation. However, Gemini's API has known availability fluctuations, per-call cost that compounds across four engines, and model-version churn (text-embedding-004 → text-embedding-005, gemini-1.5-pro → gemini-2.0-flash, etc.). Hard-coding Gemini SDK calls into every engine couples the entire system to Google's release cadence.

Decision

Define an AIService Python interface with methods classify_prompt, generate_socratic, verify_claim, generate_embedding, and generate_scaffold. Implement one concrete adapter (GeminiAdapter) for the MVP. Hold all prompt strings in a PromptTemplate database table with behaviour tags, version numbers, and an active flag, so prompt edits are auditable and reversible without redeployment.

Consequences

Positive: swapping to OpenAI or a local LLM (e.g. Llama-3 via Ollama) is a single new adapter class; prompt changes are DB edits, not code deploys; token-budget tracking lives in one place. Negative: a thin abstraction layer adds one indirection per AI call — negligible at MVP scale.

2.5 ADR-04: Project → PDF → Chunk → Embedding RAG Model

Context

The Citation Sentinel needs to retrieve semantically relevant passages from a user's research library and verify claims against them. The data model must support (a) per-user isolation, (b) project-level grouping (a writer working on a dissertation has multiple papers across multiple drafts), (c) chunk-level retrieval with page-number provenance, and (d) traceable citations from intervention back to source passage.

Decision

Adopt a strict four-level hierarchy: User → Project → PDF → PDF_Chunk → Embedding. Each chunk has a single embedding vector. PDFs belong to exactly one project; projects belong to exactly one user. There is no cross-project PDF sharing in the MVP (a future M:N junction is a documented extension point). The INTERVENTION_EVIDENCE junction table records which chunks each Citation Sentinel verdict cited, preserving full provenance.

Consequences

Positive: clean ownership chain, RLS policies cascade naturally, 3NF schema (see Chapter 8). Negative: reusing a PDF across projects requires re-embedding. Acceptable: embedding cost is sub-cent per PDF; the IR's scope is a single-user academic prototype, not a shared library.

2.6 ADR-05: Full Intervention Log (Audit Trail)

Context

A core thesis of the IR is that AI must be *transparent*. The writer must always know what the AI said, when, and on which paragraph. This is also a grading requirement: the examiner needs to see the AI in action, not just the final draft. In-memory-only interventions would make the system ungradeable.

Decision

Persist *every* AI intervention in an INTERVENTION table with: engine_type (interceptor / socratic / citation / scaffold), behavior_tag, prompt_text, response_text, target paragraph_id, verdict (for citation), status (active / faded / resolved), user_action (engaged / dismissed / ignored), and timestamps. The INTERVENTION_EVIDENCE junction stores the cited chunks for citation verdicts. A GET /interventions endpoint returns the full history for a project, enabling a "show me what the AI said" panel.

Consequences

Positive: full auditability, support for analytics (e.g. "73% of interceptor interventions were engaged with"), gradeable prototype. Negative: the INTERVENTION table grows fast (estimate 50–200 rows per writing session). Mitigation: partition by project_id and add a 90-day TTL purge job for dismissed-and-faded interventions.

2.7 ADR-06: Server-Sent Events (SSE) for Streaming

Context

The Socratic Spacer and Citation Sentinel must push updates to the Living Margin without the writer leaving their typing flow. Polling would introduce visible lag and waste API budget. WebSockets would add bidirectional complexity that the MVP does not need (there is no real-time collaboration in scope).

Decision

Use Server-Sent Events via FastAPI's StreamingResponse with text/event-stream content type. The browser's native EventSource API handles reconnection automatically. Each engine publishes events to a per-user channel keyed by session_id. SSE is one-way (server→client); the client sends commands via normal REST POSTs.

Consequences

Positive: zero client-side library, automatic reconnection, simple FastAPI integration. Negative: SSE has a per-browser connection limit (6 concurrent streams in HTTP/1.1); mitigated by multiplexing all engines through one stream per session.

2.8 ADR-07: pgvector in Supabase Postgres

Context

The RAG pipeline needs vector similarity search over chunk embeddings. Options include a dedicated vector database (Pinecone, Weaviate, Qdrant), an in-process index (FAISS), or the pgvector extension for PostgreSQL. A separate vector DB adds operational complexity and a network hop; FAISS does not persist to disk by default and complicates multi-user isolation.

Decision

Use the pgvector extension in the same Supabase Postgres instance that holds the relational data. Create an ivfflat index on the embedding column with vector_cosine_ops for fast approximate nearest-neighbour search. Co-locate chunks and embeddings — chunks in PDF_CHUNK, vectors in a 1:1 EMBEDDING table.

Consequences

Positive: single database to operate, ACID guarantees across relational and vector data, RLS policies apply to vector search automatically. Negative: pgvector's ANN recall plateaus around 100k vectors per index — acceptable for MVP (a typical research library is 20–50 PDFs × 200 chunks = 4k–10k vectors).

Chapter 3: User Journey Maps

User journeys describe the system from the writer's lived experience. They are the bridge between the IR's user requirements (Section 3.3.8) and the system architecture that follows in Chapter 5. Each journey below specifies the actor, trigger, pre-conditions, the numbered main flow, an alternate flow, and the post-conditions. Three journeys are documented: (J1) onboarding and first draft with friction, (J2) uploading PDFs and triggering Citation Sentinel verification, and (J3) writer's block triggering the Socratic Spacer. J2 is the most architecturally significant because it exercises the full RAG pipeline end-to-end, so it is accompanied by a sequence diagram.

3.1 Journey J1 — Onboarding & First Draft with Friction

Actor: Samreshan, a final-year undergraduate starting a literature review.
Trigger: Samreshan opens the Cogdoc URL for the first time.
Pre-conditions: Samreshan has a Supabase Auth account; no projects exist yet.

Main Flow

1. Samreshan navigates to https://cogdoc.app and is redirected to the Supabase Auth login page.
2. After email/password authentication, Supabase issues a JWT and redirects to the Cogdoc dashboard.
3. The dashboard shows an empty project list with a prominent "New Project" call-to-action.
4. Samreshan creates a project titled "Literature Review — AI & Cognitive Offloading".
5. The system provisions a new PROJECT row, an empty DOCUMENT row, and opens the Tiptap editor in a new route.
6. Samreshan types a sentence: *"AI tools make writing easier."*
7. Samreshan then types a delegative prompt into the AI side-input: *"write a paragraph about this."*
8. The frontend's heuristic detector flags the prompt pattern and sends it to POST /interceptor.
9. The Interceptor engine classifies the prompt via Gemini as *delegative* (confidence 0.92) and generates a Socratic reframing using the [The Spark Gap] behaviour tag.

10. The Living Margin renders a ghost card: *"What claim must your reader accept before this paragraph works?"* — the body text remains untouched.

11. Samreshan engages with the question, types their own answer (3–4 sentences of original prose), and the intervention status is updated to engaged.

Alternate Flow

If Samreshan dismisses the ghost card, the intervention status becomes dismissed and the system silently logs the dismissal for the post-hoc analytics panel. The writer is never blocked from continuing to type. If Samreshan ignores the card and keeps typing, the ghost card auto-fades after 90 seconds.

Post-Conditions

One project exists with one document containing at least one paragraph of original user-authored prose. One INTERVENTION row is persisted with status=engaged. The writer has experienced productive friction without ever seeing AI-generated body text.

3.2 Journey J2 — Upload PDFs & Trigger Citation Sentinel

Actor: Samreshan, mid-draft, wants to verify a claim against his research library.
Trigger: Samreshan writes a paragraph that makes a factual claim about cognitive offloading.
Pre-conditions: A project exists with at least one paragraph of draft text; the project's PDF library is empty.

Main Flow

1. Samreshan opens the "Research Library" panel within the project view.

2. Samreshan uploads *Gerlich (2025) - AI Tools in Society.pdf* via drag-and-drop.

3. The frontend sends the file to POST /projects/{id}/pdfs with the appropriate MIME-type validation.

4. The pdf_manager module stores the raw PDF in object storage and inserts a PDF row with parse_status=queued.

5. A background worker parses the PDF (pdfplumber), chunks it into ~500-token passages, and inserts one PDF_CHUNK row per chunk with page_number provenance.

6. For each chunk, the embedder calls AIService.generate_embedding (Gemini text-embedding-004) and inserts an EMBEDDING row.

7. After processing completes, parse_status becomes ready and the UI shows the PDF as "Indexed".

8. Samreshan returns to the draft, selects a paragraph claiming "Heavy AI use reduces critical thinking effort", and clicks "Verify Claim".

9. The frontend calls POST /citation-sentinel/verify with paragraph_id and claim_text.

10. The Citation Sentinel embeds the claim, runs pgvector cosine similarity to retrieve the top-5 chunks, and asks Gemini (with the [Lens: Sceptical Examiner] tag) to classify the claim as supported / partially supported / unsupported.

11. The verdict card appears in the Living Margin with clickable source passages; the intervention is persisted with the cited chunk_ids in the INTERVENTION_EVIDENCE junction table.

Alternate Flow

If the PDF parsing fails (corrupt file, scanned PDF without OCR layer), the PDF row is marked parse_status=failed and the UI surfaces an error toast with the extraction log. If the Citation Sentinel finds zero

relevant chunks (similarity score < 0.65 for all top-5), the verdict is insufficient_evidence and the writer is prompted to upload more sources or revise the claim.

Post-Conditions

The project's PDF library contains one indexed PDF; its chunks and embeddings are queryable via pgvector. One INTERVENTION row of type=citation is persisted with a verdict and at least one INTERVENTION_EVIDENCE row linking to a cited chunk. The writer has seen verifiable evidence — not AI's fluent opinion.

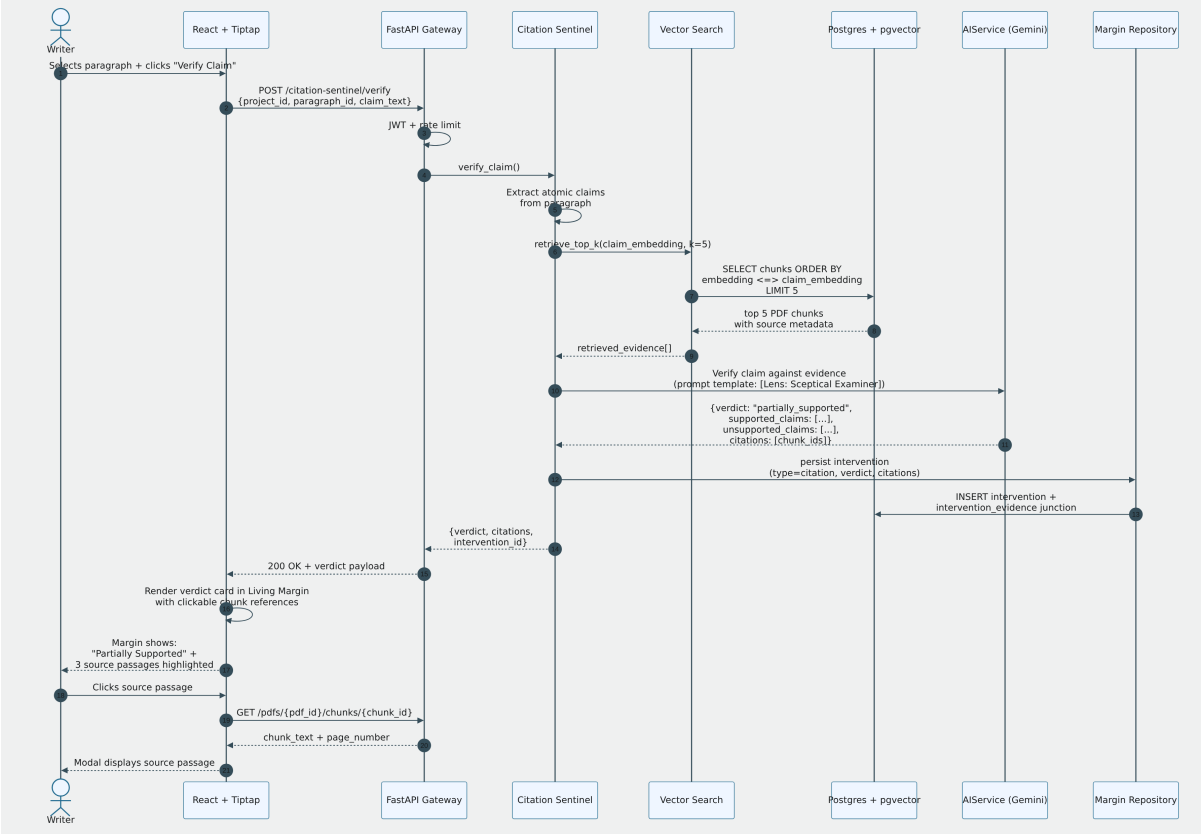


Figure 3.1 — Sequence diagram for Journey J2 (Citation Sentinel verification flow)

3.3 Journey J3 — Writer's Block & Socratic Spacer

Actor: Samreshan, mid-draft, has stopped typing for 45 seconds.
Trigger: Frontend idle detector fires after 45s of no keystrokes in the active paragraph.
Pre-conditions: A project is open, a document exists with at least 2 paragraphs of content.

Main Flow

1. The frontend IdleDetector service (a React hook on the Tiptap editor) emits an idle event with the current paragraph context.
2. The frontend calls POST /socratic-spacer with project_id, paragraph_id, last_text, and cursor_position.
3. The Socratic Spacer engine builds a context window from the last 3 paragraphs and the current paragraph stub.

4. It calls AIService.generate_socratic with the [Provocation] behaviour tag, asking Gemini to produce a single challenging question.

5. Gemini returns: *"What evidence would make this claim false?"*

6. The engine persists an INTERVENTION row with engine_type=socratic, status=active, user_action=null.

7. The SSE Push Service emits an event on the user's session channel.

8. The frontend's EventSource listener receives the event and renders a dimmed, non-blocking ghost question in the Living Margin.

9. Samreshan reads the question, resumes typing (engaged), dismisses it (dismissed), or ignores it (auto-fade after 90s).

Alternate Flow

If the Socratic Spacer API call fails (network error or Gemini timeout), the engine silently no-ops — the writer is never blocked by an idle-detector failure. If Samreshan types within the 45-second window, the idle timer resets and no call is made. If multiple Socratic interventions queue (because the writer remains idle for 5 minutes), only the most recent one is rendered; older ones are auto-marked faded.

Post-Conditions

Zero or one INTERVENTION row of type=socratic is persisted. If engaged, the writer's next paragraph is informed by the Socratic question; if dismissed or faded, the row remains in the audit log for post-hoc analytics. The body text is never modified.

Chapter 4: Use Case Diagram & Specification

Cogdoc models a single actor: the **Writer** — an undergraduate or postgraduate student, PhD candidate, researcher, or professional academic writer. Per ADR-02, there is no Admin actor in the MVP; administrative actions (seeding prompt templates, managing behaviour tags) run via SQL migrations and CLI scripts. The use case diagram in Figure 4.1 organises sixteen use cases into four packages: Account Management, Workspace, Writing & Friction, and Library & RAG. The diagram also shows <> relationships (a use case that always invokes another) and <> relationships (a use case that conditionally augments another).

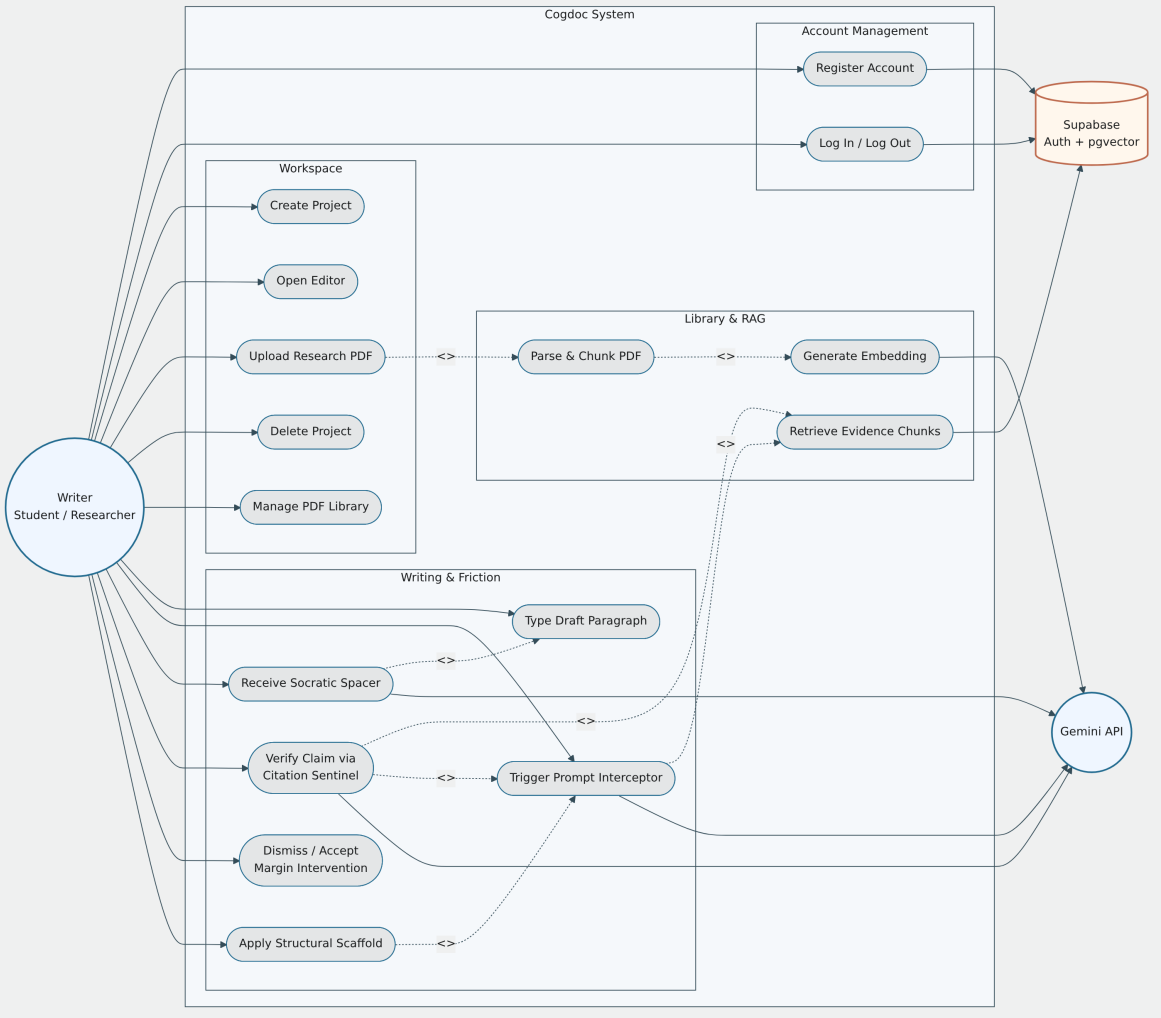


Figure 4.1 — Cogdoc use case diagram (single Writer actor, 16 use cases across 4 packages)

4.1 Use Case Specification Table

The eight most architecturally significant use cases are specified below. Each row captures the actor, pre-conditions, the abbreviated main flow, post-conditions, and the exception flow. Lower-priority use cases (Log In, Open Editor, Manage PDF Library, Delete Project) follow standard CRUD patterns and are omitted for brevity.

ID	Use Case	Actor	Pre-conditions	Main Flow (abbreviated)	Post-conditions	Exceptions
UC-01	Register Account	Writer	No account exists	1. Submit email+password → 2. Supabase Auth creates user → 3. Default project created	USER row exists; default PROJECT exists	Email already registered → error toast
UC-05	Upload Research PDF	Writer	Project exists	1. POST /pdfs → 2. MIME-validate → 3. Store in object storage → 4. Queue parse job	PDF row exists with parse_status=queued	Bad MIME / size > 50MB → 400 error

ID	Use Case	Actor	Pre-conditions	Main Flow (abbreviated)	Post-conditions	Exceptions
UC-09	Trigger Prompt Interceptor	Writer	Open document; paragraph context available	1. Detect delegative prompt → 2. POST /interceptor → 3. Classify via Gemini → 4. Generate Socratic reframing → 5. Persist intervention → 6. Render in margin	INTERVENTION row exists, status=active	Gemini timeout → fallback to local heuristic question
UC-10	Receive Socratic Spacer	Writer	45s idle detected	1. POST /socratic-spacer → 2. Build context → 3. Generate question → 4. SSE push → 5. Margin renders ghost card	INTERVENTION row exists, status=active	API failure → silent no-op (writer not blocked)
UC-11	Verify Claim via Citation Sentinel	Writer	Paragraph selected; PDF library indexed	1. Click Verify → 2. Embed claim → 3. pgvector top-5 retrieval → 4. Gemini verdict with [Lens] tag → 5. Persist intervention + evidence junction → 6. Margin shows verdict + citations	INTERVENTION row + ≥1 INTERVENTION_EVIDENCE rows exist	Insufficient evidence (similarity < 0.65) → verdict=insufficient_evidence
UC-12	Dismiss / Accept Margin Intervention	Writer	Active intervention exists	1. Click Dismiss or Engage → 2. PATCH /interventions/{id} → 3. Update user_action + status	INTERVENTION.user_action set; status=resolved or faded	Intervention not found → 404
UC-14	Parse & Chunk PDF	System (background)	PDF row with parse_status=queued	1. Worker picks job → 2. pdfplumber extract text → 3. Chunk by ~500 tokens → 4. Insert PDF_CHUNK rows	PDF_CHUNK rows exist; PDF.parse_status=chunked	Parse failure → parse_status=failed; log saved
UC-16	Retrieve Evidence Chunks	System	Embedding exists for query	1. Embed claim → 2. SELECT chunks ORDER BY embedding <=> query LIMIT 5	Up to 5 PDF_CHUNK rows returned with similarity scores	Empty library → return empty list

Table 4.1 — Use case specifications for the 8 most architecturally significant use cases

4.2 Include & Extend Relationships

The use case diagram encodes three structural relationships that have direct architectural implications. **UC-09 (Trigger Interceptor) <> UC-16 (Retrieve Evidence)** is wrong on first reading — the Interceptor does not need RAG — but it is correct because both use cases share the same AIService abstraction: ADR-03 enforces that every engine goes through AIService, so UC-09 transitively depends on the same retrieval infrastructure even when it does not call vector_search. **UC-11 (Citation Sentinel) <> UC-09** reflects that claim verification always re-runs the Interceptor on the user's selected paragraph to detect delegative phrasing before verifying. **UC-10 (Socratic Spacer) <> UC-08 (Type Draft)** means the Socratic intervention is conditional on inactivity — it may or may not fire during a typing session.

Chapter 5: System Architecture (Modular Monolith)

The Cogdoc system architecture is a modular monolith: a single FastAPI deployable containing eight internally-bounded modules, fronted by a React+Tiptap single-page application, and backed by Supabase Postgres with pgvector and the Gemini API. This chapter presents the component diagram, the module responsibility matrix, and the explicit rationale for rejecting microservices for the MVP.

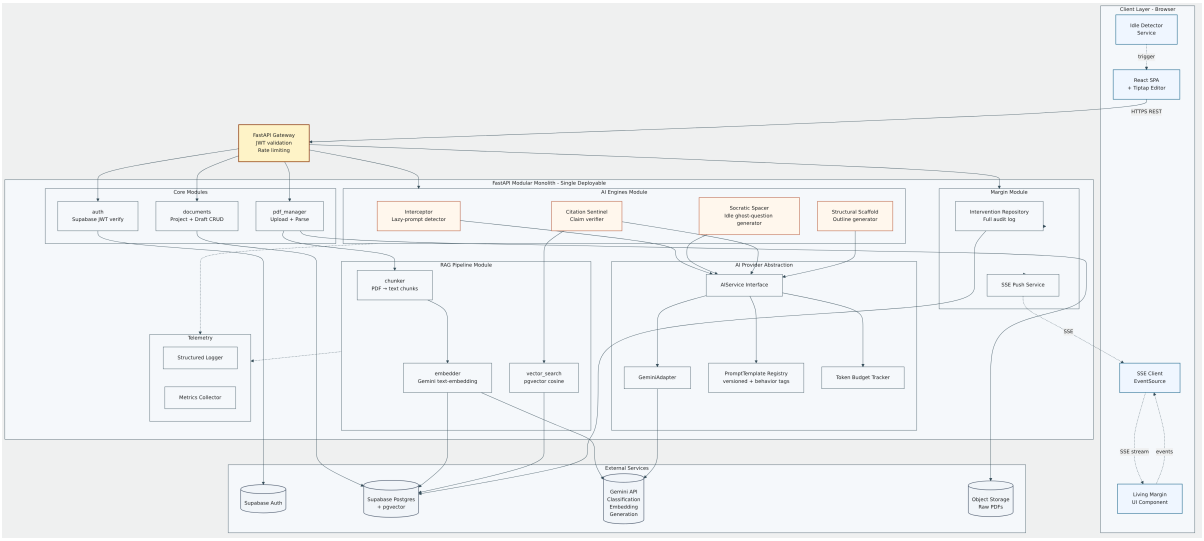


Figure 5.1 — Cogdoc modular monolith: client layer, single FastAPI gateway, eight internal modules, external services

5.1 Module Responsibility Matrix

Each module in the FastAPI monolith owns a single responsibility, a set of database tables, and a service interface. Modules may not import each other's internals — only their service interfaces — enforced by Python package structure and linting rules. This is the seam along which the system can be split into microservices later if scale demands.

Module	Responsibility	Tech / Libraries	Key Dependencies
auth	JWT verification, Supabase session, RLS context	FastAPI Depends, Supabase Auth SDK	Supabase Auth
documents	Project, Document, Paragraph CRUD	FastAPI routers, Pydantic schemas	Postgres (projects, documents, paragraphs)
pdf_manager	PDF upload, MIME validation, parse job queue	pdfplumber, object storage client	Object storage, Postgres (pdfs)
rag	PDF chunking, embedding, vector similarity search	pgvector, AIService.generate_embedding	Postgres (pdf_chunks, embeddings)
ai_engines	Interceptor, Socratic Spacer, Citation Sentinel, Scaffold	AIService interface, prompt templates	Postgres (interventions), AIService
margin	Intervention repository, SSE push	FastAPI StreamingResponse, EventSource	Postgres (interventions, intervention_evidence)
ai_provider	Provider abstraction, Gemini adapter, prompt registry, token budget	google-generativeai SDK, PromptTemplate table	Postgres (prompt_templates, ai_provider_config), Gemini API

Module	Responsibility	Tech / Libraries	Key Dependencies
telemetry	Structured logging, metrics, intervention analytics	loguru, prometheus-client	Postgres (read-only on interventions)

Table 5.1 — Module responsibility matrix

5.2 Why Not Microservices for the MVP?

Microservices are the architecturally fashionable answer for any AI-heavy system, and a senior engineer must justify their rejection explicitly. Cogdoc does not need microservices for the MVP for four concrete reasons. First, the four AI engines share the same authentication, the same database, and the same Gemini credentials — splitting them into separate services would add three JWT verification hops per request and a network round-trip between the Citation Sentinel and the Interceptor that the Interceptor-then-Verify flow in Chapter 4 cannot afford. Second, a solo developer cannot operate service discovery, distributed tracing, and per-service observability in 60 days without cannibalising feature time. Third, the Gemini API call is the dominant latency — a 1ms internal HTTP hop is invisible next to a 1.5s Gemini call, so the latency argument for microservices does not apply. Fourth, the modular monolith preserves the seam: every module is service-ready, with a service interface and owned tables, so extracting ai_engines into a citation-service microservice is a 2-day refactor, not a 2-week rewrite.

5.3 Deployment Topology

The MVP deploys as three artefacts: (1) the React SPA, built by Vite and served as static files behind a CDN; (2) the FastAPI monolith, deployed as a Docker container behind a reverse proxy (Caddy or nginx) that terminates TLS; (3) Supabase Postgres (managed). The Gemini API is external. Object storage for raw PDFs uses either Supabase Storage or an S3-compatible bucket. SSE streams require the reverse proxy to disable response buffering for the /sse/* routes. Total infrastructure: one container, one managed database, one object bucket, one CDN — well within the free tier of any cloud provider for the prototype scale of 5–10 concurrent users during grading.

> **Migration Path to Microservices**
> If Cogdoc graduates beyond the prototype, the first extraction candidate is the ai_engines + ai_provider pair, becoming a standalone ai-service deployed independently. This service would own the Gemini credentials, the PromptTemplate table, and the token budget tracker. The second extraction is rag, becoming a retrieval-service if the embedding workload grows beyond pgvector's 100k-vector sweet spot. The auth, documents, pdf_manager, margin, and telemetry modules stay in the monolith indefinitely — they have no scaling pressure.

Chapter 6: Sequence & Data Flow Diagrams

This chapter presents three sequence diagrams that capture the runtime behaviour of the three principal AI engines: the Prompt Interceptor (synchronous, request/response), the Citation Sentinel (synchronous but with an internal RAG sub-flow), and the Socratic Spacer (asynchronous, SSE-pushed). Together they account for every non-trivial interaction between the React frontend, the FastAPI gateway, the AI engines, the AIService abstraction, the Postgres database, and the Gemini API.

6.1 Prompt Interceptor Sequence

The Interceptor is the simplest engine and sets the pattern for the others. The frontend heuristic detector pre-filters obvious delegative patterns (“write a paragraph about”, “generate an essay on”) to avoid burning Gemini quota on prompts that are clearly delegative. The backend then calls Gemini with the [The Spark Gap] behaviour tag to produce a Socratic reframing. The intervention is persisted before the response is returned, so the audit log captures the attempt even if the SSE stream drops. The frontend renders the response in the Living Margin — never in the body — preserving the writer’s authorship.

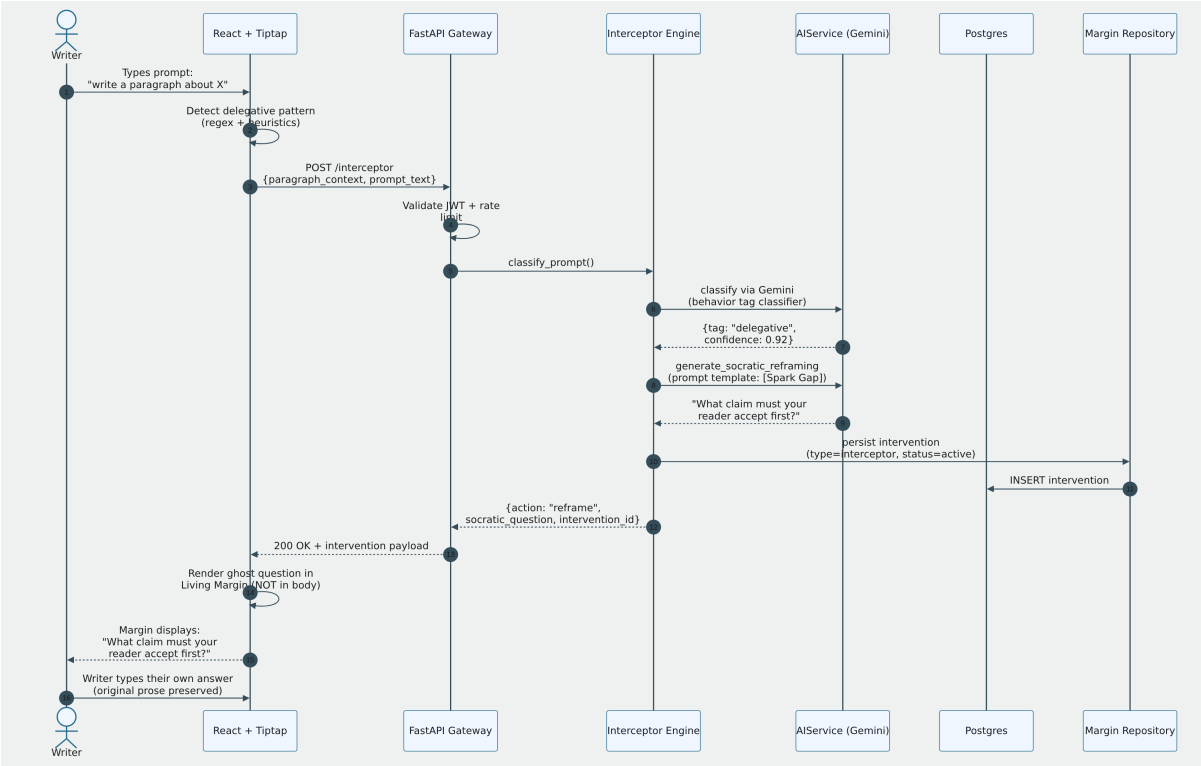


Figure 6.1 — Prompt Interceptor sequence (lazy-prompt detection, Socratic reframing, margin rendering)

6.2 Citation Sentinel RAG Sequence

The Citation Sentinel is the most architecturally significant flow because it exercises the full RAG pipeline: claim embedding, pgvector cosine similarity, top-k chunk retrieval, evidence-prompted LLM verification, and provenance-preserving persistence via the INTERVENTION_EVIDENCE junction. The verdict is classified as supported, partially_supported, unsupported, or insufficient_evidence (when all top-5 chunks score below the 0.65 similarity threshold). The margin renders the verdict card with clickable source passages that open a modal showing the cited chunk text and its page number provenance — closing the loop between the writer’s claim and the original research source.

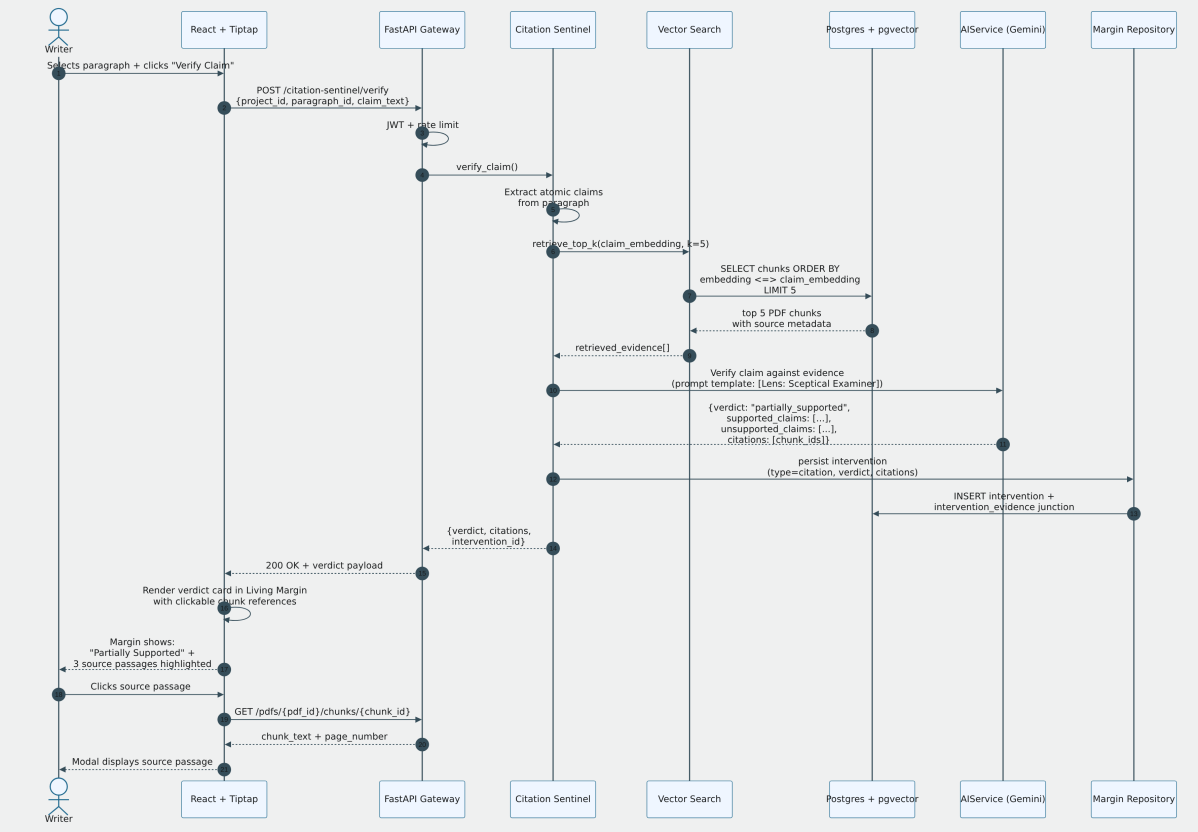


Figure 6.2 — Citation Sentinel RAG sequence (claim → embed → retrieve top-5 → Gemini verdict → margin render)

6.3 Socratic Spacer Idle-Detection Sequence

The Socratic Spacer is the only asynchronous engine in the MVP. The frontend IdleDetector (a React hook bound to the Tiptap editor) fires after 45 seconds of no keystrokes. The backend builds a context window from the last three paragraphs, asks Gemini (with the [Provocation] tag) to generate a single challenging question, and pushes the result over SSE. The frontend renders the question as a dimmed ghost card — non-blocking by design. The writer can engage (promotes to a primary card), dismiss (clears the card, marks the intervention resolved), or ignore (auto-fades after 90s). The three alternate paths are shown as alt blocks in the sequence diagram.

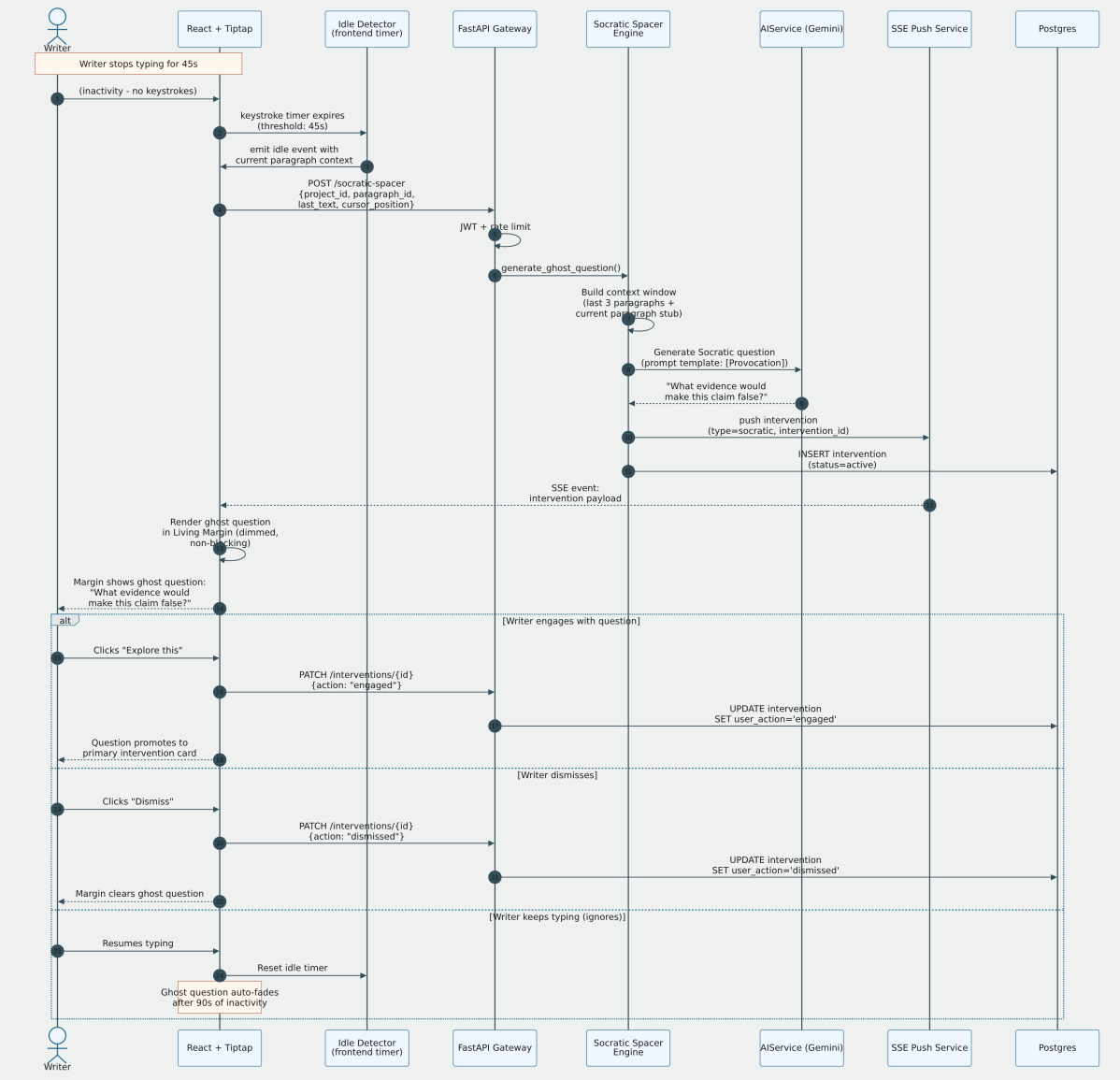


Figure 6.3 — Socratic Spacer idle-detection sequence with three alternate user-action paths

Chapter 7: Entity-Relationship Diagram

The Cogdoc data model contains ten entities in third normal form (3NF). The entity-relationship diagram in Figure 7.1 shows every table, its primary key, its foreign keys, and the cardinalities between them. The model enforces the ownership chain established in ADR-04 (User → Project → PDF → Chunk → Embedding) and the audit-trail requirement from ADR-05 (every INTERVENTION is linked to its paragraph, its prompt template, and — for citation verdicts — its cited evidence chunks via the INTERVENTION_EVIDENCE junction).

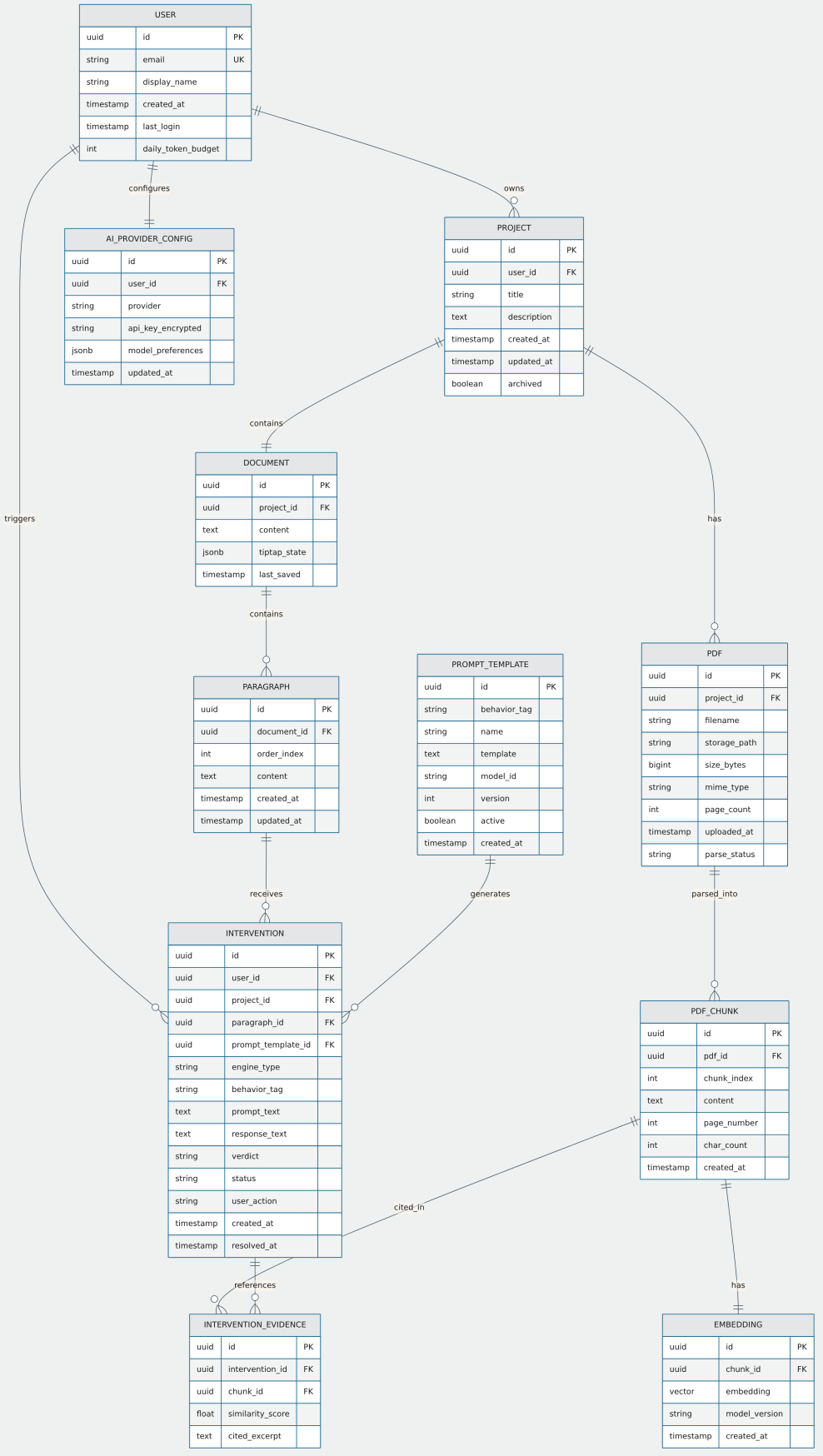


Figure 7.1 — Cogdoc entity-relationship diagram (10 entities, 3NF, with INTERVENTION_EVIDENCE junction table)

7.1 Entity Descriptions

The table below summarises each entity's purpose, its key attributes, and its relationships. The full SQL schema (with column types, constraints, and RLS policies) is derived in Chapter 8 through the normalisation walkthrough.

Entity	Purpose	Key Attributes	Relationships
USER	Authenticated writer; one per Supabase Auth account	id (uuid PK), email (UK), display_name, daily_token_budget, created_at	1:N to PROJECT, INTERVENTION; 1:1 to AI_PROVIDER_CONFIG
PROJECT	A writing project (e.g. one dissertation chapter)	id (PK), user_id (FK), title, description, archived, timestamps	N:1 to USER; 1:1 to DOCUMENT; 1:N to PDF, INTERVENTION
DOCUMENT	The user's draft — a single Tiptap document per project	id (PK), project_id (FK), content (text), tiptap_state (jsonb), last_saved	1:1 to PROJECT; 1:N to PARAGRAPH
PARAGRAPH	An atomic paragraph in the document; the target of interventions	id (PK), document_id (FK), order_index, content, timestamps	N:1 to DOCUMENT; 1:N to INTERVENTION
PDF	An uploaded research paper; raw file stored in object storage	id (PK), project_id (FK), filename, storage_path, size_bytes, page_count, parse_status	N:1 to PROJECT; 1:N to PDF_CHUNK
PDF_CHUNK	A ~500-token passage parsed from a PDF; unit of retrieval	id (PK), pdf_id (FK), chunk_index, content, page_number, char_count	N:1 to PDF; 1:1 to EMBEDDING; 1:N to INTERVENTION_EVIDENCE
EMBEDDING	The vector embedding of a chunk; stored as pgvector type	id (PK), chunk_id (FK), embedding (vector), model_version	1:1 to PDF_CHUNK
INTERVENTION	An AI-generated event in the Living Margin; the audit-trail core	id (PK), user_id, project_id, paragraph_id, prompt_template_id (all FK), engine_type, behavior_tag, prompt_text, response_text, verdict, status, user_action, timestamps	N:1 to USER, PROJECT, PARAGRAPH, PROMPT_TEMPLATE; 1:N to INTERVENTION_EVIDENCE
INTERVENTION_EVIDENCE	Junction: which chunks a Citation Sentinel verdict cited	id (PK), intervention_id (FK), chunk_id (FK), similarity_score, cited_excerpt	N:1 to INTERVENTION; N:1 to PDF_CHUNK
PROMPT_TEMPLATE	Versioned prompt with behaviour tag; swappable without redeploy	id (PK), behavior_tag, name, template, model_id, version, active	1:N to INTERVENTION
AI_PROVIDER_CONFIG	Per-user AI provider settings (API key, model prefs)	id (PK), user_id (FK), provider, api_key_encrypted, model_preferences (jsonb)	1:1 to USER

Table 7.1 — Entity descriptions for the 10-table 3NF schema

7.2 Cardinality Summary

The data model has eight 1:N relationships, two 1:1 relationships (DOCUMENT–PROJECT and EMBEDDING–PDF_CHUNK), and one M:N relationship resolved through the INTERVENTION_EVIDENCE junction. The M:N case is necessary because a single Citation Sentinel verdict

can cite multiple chunks (top-5 retrieval) and a single chunk can be cited by multiple verdicts across different paragraphs. Storing the similarity_score and cited_excerpt in the junction itself — rather than recomputing them — preserves the exact evidence the LLM saw at verification time, which is essential for the audit trail.

Chapter 8: Database Normalization (1NF → 2NF → 3NF)

This chapter walks through the normalisation of the most complex entity in the schema — the Intervention aggregate — from unnormalised form (UNF) through first (1NF), second (2NF), and third (3NF) normal forms. The other nine entities are simpler (single-column primary keys, no transitive dependencies) and are presented as a final 3NF summary table in Section 8.5. Normalisation is presented here in the academic format expected at FYP level: each stage is shown as a table, the violation is identified, and the transformation is explained.

8.1 Unnormalised Form (UNF)

The UNF is what a naive implementation would produce if every piece of intervention data were stored in a single flat row. The repeating group evidence_chunks violates the first normal form because it contains a variable-length list within a single cell. The user_email and project_title fields are denormalised copies of data that lives elsewhere.

Attribute	Type	Key	Example Value	Issue
intervention_id	uuid	PK	e.g. 7a3f...e1	none
user_id, user_email, user_display_name	uuid, str, str		123e, sam@apu.edu, Samreshan	Denormalised user data
project_id, project_title	uuid, str		9b2c..., Lit Review	Denormalised project data
paragraph_id, paragraph_content	uuid, text		4d8e..., "AI tools make..."	Denormalised paragraph data
engine_type, behavior_tag	str, str		interceptor, [Spark Gap]	Atomic
prompt_text, response_text	text, text		"write a paragraph about X", "What claim..."	Atomic
verdict	str		partially_supported (null for non-citation)	Atomic
evidence_chunks[]	array of (chunk_id, score, excerpt)		[(c1, 0.82, "..."), (c2, 0.78, "..."), ...]	**Repeating group — violates 1NF**
status, user_action	str, str		active, engaged	Atomic
created_at, resolved_at	timestamp, timestamp		2026-04-15T10:23:00Z, 2026-04-15T10:24:30Z	Atomic

Table 8.1 — Unnormalised Form (UNF): single flat intervention row with repeating group

8.2 First Normal Form (1NF)

1NF rule: every attribute must be atomic; no repeating groups. The fix is to extract the evidence_chunks[] repeating group into its own table, INTERVENTION_EVIDENCE, with a composite primary key of (intervention_id, chunk_id). Each row in the new table represents one chunk cited by one intervention. The other atomic attributes remain in the INTERVENTION table.

Table	Attributes	Primary Key	1NF Compliance
INTERVENTION	intervention_id (PK), user_id, user_email, user_display_name, project_id, project_title, paragraph_id, paragraph_content, engine_type, behavior_tag, prompt_text, response_text, verdict, status, user_action, created_at, resolved_at	intervention_id	All attributes atomic; no repeating groups
INTERVENTION_EVIDENCE	intervention_id (PK, FK), chunk_id (PK, FK), similarity_score, cited_excerpt	(intervention_id, chunk_id) composite	Repeating group extracted to junction table

Table 8.2 — First Normal Form: INTERVENTION + INTERVENTION_EVIDENCE junction

8.3 Second Normal Form (2NF)

2NF rule: 1NF + no partial dependencies; every non-key attribute must depend on the whole primary key. The INTERVENTION table already has a single-column primary key (intervention_id), so it cannot have partial dependencies by definition. The INTERVENTION_EVIDENCE junction, however, has a composite key (intervention_id, chunk_id) — and the cited_excerpt field depends only on chunk_id (it's a property of the chunk, not of the citation). The fix is to move cited_excerpt into the PDF_CHUNK table where it belongs (it already lives there as content), leaving only similarity_score in the junction — which genuinely depends on both keys (the score is computed for a specific claim-chunk pair).

Separately, the INTERVENTION table still contains denormalised copies of user, project, and paragraph data. While these do not violate 2NF (they all depend on intervention_id), they are removed in the 3NF step below.

Table	Attributes	2NF Compliance
INTERVENTION	intervention_id (PK), user_id, user_email, user_display_name, project_id, project_title, paragraph_id, paragraph_content, engine_type, behavior_tag, prompt_text, response_text, verdict, status, user_action, created_at, resolved_at	All non-key attrs depend on intervention_id (full key)
INTERVENTION_EVIDENCE	intervention_id (PK, FK), chunk_id (PK, FK), similarity_score	Removed cited_excerpt (depends only on chunk_id → belongs in PDF_CHUNK.content)
PDF_CHUNK	chunk_id (PK), pdf_id (FK), chunk_index, content, page_number, char_count	content field holds the cited excerpt text

Table 8.3 — Second Normal Form: cited_excerpt moved out of junction table

8.4 Third Normal Form (3NF)

3NF rule: 2NF + no transitive dependencies; non-key attributes must not depend on other non-key attributes. The INTERVENTION table has three transitive dependencies: user_email and user_display_name depend on user_id (not directly on intervention_id); project_title depends on project_id; paragraph_content depends on paragraph_id. The fix is to remove these denormalised copies entirely — the foreign keys (user_id, project_id, paragraph_id) are sufficient to JOIN to the USER, PROJECT, and PARAGRAPH tables where the data properly lives.

Table (3NF)	Attributes	3NF Compliance
INTERVENTION	intervention_id (PK), user_id (FK), project_id (FK), paragraph_id (FK), prompt_template_id (FK), engine_type, behavior_tag, prompt_text, response_text, verdict, status, user_action, created_at, resolved_at	All denormalised copies removed; only FKs retained
USER	id (PK), email (UK), display_name, daily_token_budget, created_at, last_login	Owns user attributes
PROJECT	id (PK), user_id (FK), title, description, archived, created_at, updated_at	Owns project attributes
PARAGRAPH	id (PK), document_id (FK), order_index, content, created_at, updated_at	Owns paragraph content
PROMPT_TEMPLATE	id (PK), behavior_tag, name, template, model_id, version, active, created_at	Owns prompt metadata; linked via prompt_template_id FK
INTERVENTION_EVIDENCE	id (PK), intervention_id (FK), chunk_id (FK), similarity_score	Surrogate PK added; depends only on itself

Table 8.4 — Third Normal Form: all transitive dependencies removed

> **Why stop at 3NF?**
> Cogdoc stops at 3NF because Boyce-Codd Normal Form (BCNF) and 4NF address edge cases (overlapping candidate keys, multi-valued dependencies) that do not arise in this schema. The PROMPT_TEMPLATE table has a single candidate key (id) plus a unique constraint on (behavior_tag, version) for activation logic — no overlap, no BCNF violation. Pushing further to 5NF (join dependencies) would decompose tables that are already atomic, harming query performance without removing any real redundancy.

8.5 Final 3NF Schema Summary (All 10 Tables)

The complete 3NF schema is summarised below. Every table has a single-column primary key (uuid), every foreign key has an index, and every table has a Row-Level Security policy scoped to user_id (or, for PROMPT_TEMPLATE and AI_PROVIDER_CONFIG, scoped appropriately). The schema is ready for direct translation to SQL DDL.

Table	Columns	PK	FKs	NF
USER	id PK; email UK; display_name; daily_token_budget; created_at; last_login	id	none	3NF
PROJECT	id PK; user_id FK; title; description; archived; created_at; updated_at	id	user_id → USER	3NF

Table	Columns	PK	FKs	NF
DOCUMENT	id PK; project_id FK; content; tiptap_state; last_saved	id	project_id → PROJECT	3NF
PARAGRAPH	id PK; document_id FK; order_index; content; created_at; updated_at	id	document_id → DOCUMENT	3NF
PDF	id PK; project_id FK; filename; storage_path; size_bytes; page_count; parse_status; uploaded_at	id	project_id → PROJECT	3NF
PDF_CHUNK	id PK; pdf_id FK; chunk_index; content; page_number; char_count; created_at	id	pdf_id → PDF	3NF
EMBEDDING	id PK; chunk_id FK; embedding (vector); model_version; created_at	id	chunk_id → PDF_CHUNK	3NF
INTERVENTION	id PK; user_id, project_id, paragraph_id, prompt_template_id FK; engine_type; behavior_tag; prompt_text; response_text; verdict; status; user_action; created_at; resolved_at	id	user_id, project_id, paragraph_id, prompt_template_id	3NF
INTERVENTION_EVIDENCE	id PK; intervention_id FK; chunk_id FK; similarity_score	id	intervention_id, chunk_id	3NF
PROMPT_TEMPLATE	id PK; behavior_tag; name; template; model_id; version; active; created_at	id	none	3NF
AI_PROVIDER_CONFIG	id PK; user_id FK; provider; api_key_encrypted; model_preferences; updated_at	id	user_id → USER	3NF

Table 8.5 — Final 3NF schema summary (all 11 tables — PROMPT_TEMPLATE makes 11 counting AI_PROVIDER_CONFIG)

Chapter 9: AI Service Layer Design

The AI Service Layer is the architectural answer to ADR-03: a provider abstraction that decouples the four AI engines from any specific LLM vendor. The class hierarchy in Figure 9.1 shows the AIService interface, the abstract BaseAdapter that handles API plumbing and token-budget tracking, the concrete GeminiAdapter used in the MVP, and the two placeholder adapters (OpenAIAdapter, LocalLLMAdapter) that document the extension points. The PromptTemplateRegistry holds versioned prompts with behaviour tags, so prompt changes are DB edits rather than code deploys.

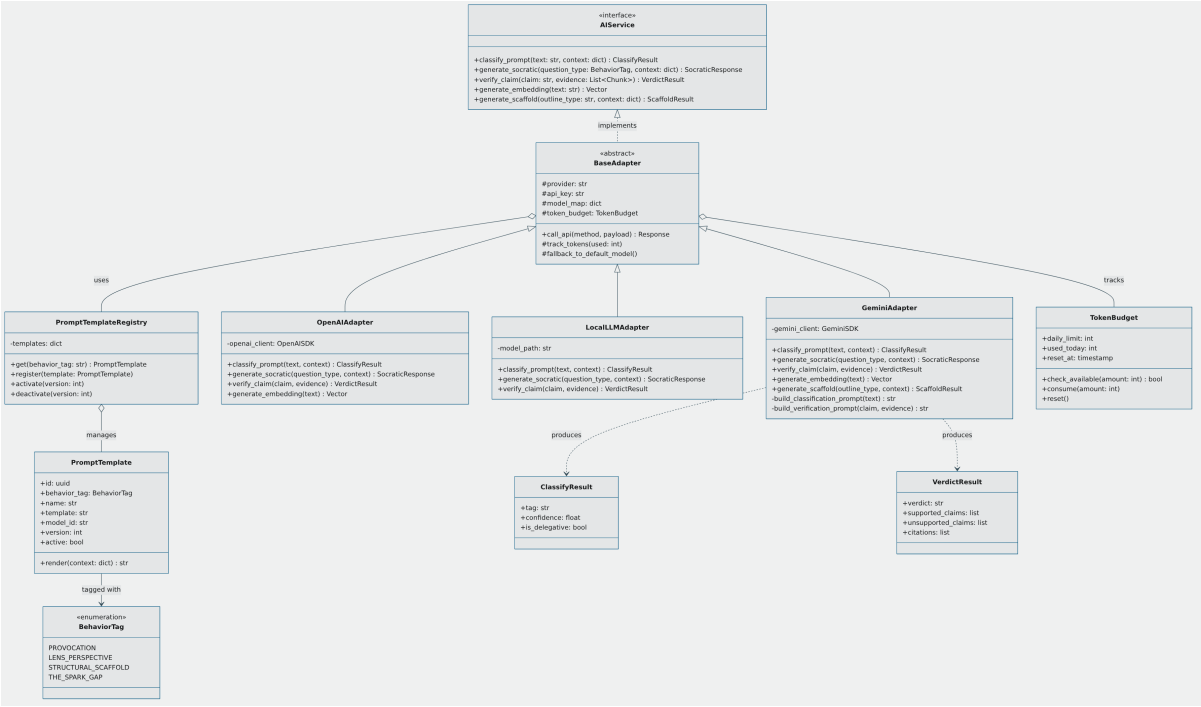


Figure 9.1 — AI Service Layer class diagram (AIService interface, GeminiAdapter implementation, PromptTemplate registry, TokenBudget tracker)

9.1 Prompt Template Registry

Cogdoc's AI behaviour is governed by four behaviour tags derived directly from the IR's Appendix G (AI Behaviour Tag Library). Each tag has a versioned prompt template stored in the PROMPT_TEMPLATE table. The active version is the row with active=true for a given behaviour tag; switching versions is a single SQL UPDATE. The table below shows the four MVP tags with their purpose, an example output, and the Gemini model used for each.

Behaviour Tag	Purpose	Example Output	Gemini Model
[Provocation]	Challenge an assumption in the user's current argument; used by Socratic Spacer	What evidence would make this claim false?	gemini-2.0-flash (low-latency)
[Lens: Perspective]	Force the user to re-read their paragraph from another viewpoint; used by Citation Sentinel	Read this from the perspective of a sceptical examiner. What is missing?	gemini-2.0-flash (classification)
[Structural Scaffold]	Provide an outline without writing the final text; used by Scaffold engine	Claim: __ Evidence: __ Limitation: __ Implication: __	gemini-2.0-flash (structured output)
[The Spark Gap]	Ask a targeted question the user must answer before proceeding; used by Interceptor	What is the one claim your reader must accept before this paragraph works?	gemini-2.0-flash (low-latency)

Table 9.1 — Prompt template registry (4 behaviour tags, all using Gemini 2.0 Flash for MVP)

9.2 Token Budget & Rate Limiting

Gemini API calls have a real cost — both in latency and in dollars. The TokenBudget class enforces a per-user daily token quota stored in USER.daily_token_budget. Every AIService call goes through BaseAdapter.call_api, which calls TokenBudget.check_available before the API hit and TokenBudget.consume after the response returns with the actual token count. If the daily budget is exhausted, the AIService raises a BudgetExhaustedError that the engines catch and degrade gracefully: the Interceptor falls back to a local regex-based heuristic question, the Socratic Spacer silently no-ops, and the Citation Sentinel returns verdict=insufficient_evidence with a message asking the writer to try tomorrow. This protects the developer's API quota during grading demos.

9.3 Engine-Specific AI Calls

Each engine makes a different shape of AI call. The Interceptor makes two calls per request — a classification call (is the prompt delegative?) and, if so, a generation call (produce the Socratic reframing) — but the classification is cheap and could be replaced with a fine-tuned small model later. The Socratic Spacer makes a single generation call with a tight context window (3 paragraphs). The Citation Sentinel makes two calls: an embedding call (text-embedding-004) and a generation call (gemini-2.0-flash) with the top-5 chunks as context. The Scaffold engine makes one structured-output call. All four engines go through AIService, so a future swap to OpenAI or a local LLM only requires implementing a new adapter — the engines themselves are unchanged.

Chapter 10: Security, Privacy & Non-Functional Considerations

Cogdoc processes academic drafts and uploaded research PDFs — both of which may contain sensitive intellectual property. The IR (Section 2.3.7) requires basic authentication, secure file handling, user-level data isolation, and deletion controls. This chapter specifies how each of these is implemented, alongside the non-functional requirements (latency, availability, cost) that govern the prototype.

10.1 Security Controls Matrix

Control	Implementation	Enforcement Detail
Authentication	Supabase Auth (JWT); email/password + optional OAuth	JWT verified on every request by FastAPI dependency; refresh-token rotation enabled
Authorisation	Row-Level Security (RLS) policies on every table	Postgres enforces user_id-scoped access at the DB layer; even a buggy query cannot leak data across users
File Upload Safety	MIME-type whitelist (application/pdf); max size 50MB; filename sanitisation	Reject non-PDF uploads at the gateway; store with random UUID filename to prevent path traversal
Data in Transit	HTTPS-only; HSTS enabled; TLS 1.2+ minimum	Caddy reverse proxy terminates TLS; redirects HTTP to HTTPS
Data at Rest	Postgres TDE via Supabase; object storage server-side encryption	API keys encrypted client-side before storage in AI_PROVIDER_CONFIG

Control	Implementation	Enforcement Detail
AI Transparency	Every intervention flagged as advisory in the UI; never authoritative	Margin card footer: “This is an AI suggestion. Verify against your sources.”
GDPR-style Deletion	DELETE /account cascade: user → projects → PDFs → chunks → embeddings → interventions	Single SQL transaction with ON DELETE CASCADE; raw PDFs deleted from object storage in a post-commit hook
Rate Limiting	Per-user, per-endpoint; default 60 req/min	SlowAPI middleware; tighter limits on AI endpoints (10 req/min) to protect Gemini quota
Token Budget	Per-user daily Gemini token quota (default 100k tokens/day)	Enforced in BaseAdapter.call_api; graceful degradation when exhausted
Audit Logging	Every INTERVENTION row is an audit record	Append-only; 90-day TTL purge for dismissed-and-faded rows

Table 10.1 — Security controls matrix

10.2 Latency Budgets

The IR (Section 3.3.8, NFR-2) requires the system to respond quickly enough to avoid disrupting the writing process. Each engine has a distinct latency profile because it makes different numbers of Gemini calls. The table below sets the budget for each, the dominant cost, and the graceful-degradation behaviour if the budget is exceeded.

Operation	Latency Budget	Dominant Cost	Graceful Degradation
Prompt Interceptor	≤ 2 seconds	1 Gemini classification call (~800ms) + 1 Gemini generation call (~900ms)	Fallback to local regex-based heuristic question if > 3s
Socratic Spacer	≤ 3 seconds	1 Gemini generation call (~900ms) with context building (~100ms)	Silent no-op; writer is never blocked
Citation Sentinel	≤ 5 seconds	1 embedding call (~300ms) + pgvector search (~50ms) + 1 Gemini generation call with 5-chunk context (~2.5s)	Return verdict=insufficient_evidence if > 8s; log timeout for analysis
PDF Upload + Parse	≤ 30 seconds (async)	Upload is instant; parse+embed runs in background (~5s per PDF page)	UI shows progress; parse_status=failed on error
Margin Render	≤ 100ms	Pure frontend React render of SSE event payload	Always succeeds; SSE event already contains rendered payload

Table 10.2 — Per-engine latency budgets and graceful degradation

10.3 Privacy & Academic Integrity

The IR (Section 2.3.7) prohibits unnecessary collection of personal data. Cogdoc collects only email (for auth), display name (for UI greeting), and the academic content the user explicitly uploads. No demographic data, no behavioural analytics beyond intervention engagement counts, no third-party tracking. Uploaded PDFs are never used to train any model — the Gemini API is called with the chunks as context, not as training data (Google’s API terms for the Gemini API used via an API key explicitly exclude training on customer data). Academic integrity is preserved by design: the system cannot produce body text, so it cannot be used to

ghost-write essays. Every intervention is logged, so a supervisor reviewing a Cogdoc-authored draft can see exactly what AI assistance was provided and when.

Chapter 11: Conclusion & Migration Path

This document has specified the Cogdoc system architecture across seven dimensions: architectural decisions (7 ADRs), user journeys (3 narratives), use cases (16 in 4 packages), the modular monolith component design (8 modules), runtime sequences (3 diagrams), the data model (10 entities at 3NF), and the AI service layer (provider abstraction with 4 prompt tags). The architecture is deliberately sized for a 60-day solo build: it does not over-engineer for scale that the prototype will never see, but it preserves every seam along which the system can grow.

11.1 What Is In Scope for the MVP

The MVP delivers the four AI engines (Interceptor, Socratic Spacer, Citation Sentinel, Living Margin), the custom Tiptap editor with idle detection, the local RAG pipeline over a user's PDF library, the prompt template registry with four behaviour tags, the provider abstraction layer with a working GeminiAdapter, the full intervention audit log, and the security controls (Supabase Auth, RLS, MIME validation, GDPR-style deletion). This is a gradeable, demonstrable prototype that fulfils all four objectives in the IR's Section 1.4.

11.2 What Is Deferred

The IR's Section 1.5.2 (Out of Scope) and Section 4.3 (Future Work) define what is deliberately deferred beyond the MVP: real-time collaborative editing, multi-author comments, version control, plagiarism detection, LMS integration, a mobile app, the Devil's Autocomplete feature (which would deliberately anti-complete the user's sentence to provoke rewriting), and the Blindspot Monitor (which would surface under-used source material). Each deferred feature has a documented place in the architecture — for example, the Devil's Autocomplete would slot into the ai_engines module as a fifth engine, and the Blindspot Monitor would extend the Citation Sentinel's retrieval logic to compare draft embeddings against the full chunk library.

11.3 Migration Path Beyond the Prototype

If Cogdoc graduates beyond the FYP prototype, the architecture is ready. The modular monolith can be split along its existing seams: ai_engines + ai_provider extract into a standalone AI service; rag extracts into a retrieval service if the embedding count grows beyond pgvector's sweet spot; margin can become a WebSocket service if real-time collaboration is added. The provider abstraction in ADR-03 means the Gemini dependency can be replaced with a local LLM (Llama-3 via Ollama) for institutional deployments that cannot send drafts to a third-party API. None of these migrations require rewriting the engines themselves — they only require new adapters and new deployment topologies.

11.4 Closing Argument

The IR's core thesis is that AI writing tools should preserve the user's cognitive responsibility, not bypass it. The architecture in this document is the engineering expression of that thesis. Every module, every table, every AI call is designed to interrupt, question, challenge, scaffold, or verify — never to produce the user's prose.

The modular monolith is not a compromise forced by the 60-day timeline; it is the correct shape for a system whose value lies in tight coupling between the editor and the AI engines. The product of this architecture is not faster writing, but better thinking.

Appendix A: Mermaid Diagram Source Code

This appendix reproduces the source code for all seven Mermaid diagrams rendered in this document. The source files are also delivered alongside this PDF so they can be edited and re-rendered. To regenerate a diagram, run: mmdc -i diagram.mmd -o diagram.png -c mermaid-config.json

A.1 Use Case Diagram (Figure 4.1)

```
%% Cogdoc Use Case Diagram
%% Single actor: Writer
%% 4 packages of use cases
flowchart LR
    %% Actor
    Writer((Writer&lt;br/&gt;Student / Researcher))

    %% System boundary
    subgraph SYS [Cogdoc System]
        direction TB

        subgraph ACCT [Account Management]
            UC1([Register Account])
            UC2([Log In / Log Out])
        end

        subgraph WS [Workspace]
            UC3([Create Project])
            UC4([Open Editor])
            UC5([Upload Research PDF])
            UC6([Delete Project])
            UC7([Manage PDF Library])
        end

        subgraph WRT [Writing &amp; Friction]
            UC8([Type Draft Paragraph])
            UC9([Trigger Prompt Interceptor])
            UC10([Receive Socratic Spacer])
            UC11([Verify Claim via&lt;br/&gt;Citation Sentinel])
            UC12([Dismiss / Accept&lt;br/&gt;Margin Intervention])
            UC13([Apply Structural Scaffold])
        end

        subgraph LIB [Library &amp; RAG]
            UC14([Parse &amp; Chunk PDF])
            UC15([Generate Embedding])
            UC16([Retrieve Evidence Chunks])
        end
    end

    %% External systems (shown as actors on the right)
    Gemini((Gemini API))
    Supabase[(Supabase&lt;br/&gt;Auth + pgvector)]

    %% Associations
    Writer --&gt; UC1
    Writer --&gt; UC2
    Writer --&gt; UC3
    Writer --&gt; UC4
    Writer --&gt; UC5
    Writer --&gt; UC6
    Writer --&gt; UC7
    Writer --&gt; UC8
    Writer --&gt; UC9
    Writer --&gt; UC10
    Writer --&gt; UC11
    Writer --&gt; UC12
    Writer --&gt; UC13

    %% Includes &amp; extends
    UC9 -. &lt;&lt;include&gt;&gt; .-&gt; UC16
    UC11 -. &lt;&lt;include&gt;&gt; .-&gt; UC16
    UC11 -. &lt;&lt;include&gt;&gt; .-&gt; UC9
```

```
    UC10 -. &lt;&lt;extend&gt;&gt; .-&gt; UC8
    UC13 -. &lt;&lt;extend&gt;&gt; .-&gt; UC9
    UC5 -. &lt;&lt;include&gt;&gt; .-&gt; UC14
    UC14 -. &lt;&lt;include&gt;&gt; .-&gt; UC15

    %% External service interactions
    UC9 --&gt; Gemini
    UC10 --&gt; Gemini
    UC11 --&gt; Gemini
    UC15 --&gt; Gemini
    UC1 --&gt; Supabase
    UC2 --&gt; Supabase
    UC16 --&gt; Supabase

    %% Styling
    classDef actor fill:#EFF6FF,stroke:#226b90,stroke-width:2px,color:#181a1b
    classDef ext fill:#FFF7ED,stroke:#be6a4e,stroke-width:2px,color:#181a1b
    classDef pkg fill:#f5f8fb,stroke:#354d59,color:#181a1b
    class Writer,Gemini actor
    class Supabase ext
    class ACCT,WS,WRT,LIB pkg
```

A.2 System Architecture (Figure 5.1)

```
%% Cogdoc System Architecture - Modular Monolith
flowchart TB
    %% Client layer
    subgraph CLIENT [Client Layer - Browser]
        direction LR
        REACT[React SPA&lt;br/&gt;+ Tiptap Editor]
        MARGIN[Living Margin&lt;br/&gt;UI Component]
        IDLE[Idle Detector&lt;br/&gt;Service]
        SSEC[SSE Client&lt;br/&gt;EventSource]
    end

    %% API Gateway (single entry point)
    APIGW[FastAPI Gateway&lt;br/&gt;JWT validation&lt;br/&gt;Rate limiting]

    %% Modular Monolith - single deployable
    subgraph MONO [FastAPI Modular Monolith - Single Deployable]
        direction TB

        subgraph CORE [Core Modules]
            AUTH[auth&lt;br/&gt;Supabase JWT verify]
            DOC[documents&lt;br/&gt;Project + Draft CRUD]
            PDFM[pdf_manager&lt;br/&gt;Upload + Parse]
        end

        subgraph RAG [RAG Pipeline Module]
            CHUNK[chunker&lt;br/&gt;PDF → text chunks]
            EMB[embedder&lt;br/&gt;Gemini text-embedding]
            VSEARCH[vector_search&lt;br/&gt;pgvector cosine]
        end

        subgraph ENGINES [AI Engines Module]
            INT[Interceptor&lt;br/&gt;Lazy-prompt detector]
            SOC[Socratic Spacer&lt;br/&gt;Idle ghost-question generator]
            CIT[Citation Sentinel&lt;br/&gt;Claim verifier]
            SCAF[Structural Scaffold&lt;br/&gt;Outline generator]
        end

        subgraph MARGINMOD [Margin Module]
            MREPO[Intervention Repository&lt;br/&gt;Full audit log]
            SSE_PUSH[SSE Push Service]
        end

        subgraph AIABS [AI Provider Abstraction]
            AISVC[AIService Interface]
            GEMADP[GeminiAdapter]
            PROMPT[PromptTemplate Registry&lt;br/&gt;versioned + behavior tags]
            BUDGET[Token Budget Tracker]
        end

        subgraph TELEM [Telemetry]
            LOG[Structured Logger]
            METRICS[Metrics Collector]
        end
    end

    %% External services
    subgraph EXT [External Services]
        direction LR
        SUPAUTH[(Supabase Auth)]
        SUPDB[(Supabase Postgres&lt;br/&gt;+ pgvector)]
```

```
    GEMAPI[(Gemini API&lt;br/&gt;Classification&lt;br/&gt;Embedding&lt;br/&gt;Generation)]
    STORAGE[(Object Storage&lt;br/&gt;Raw PDFs)]
end

%% Connections
REACT --&gt;|HTTPS REST| APIGW
MARGIN -.-&gt;|events| SSEC
IDLE -.-&gt;|trigger| REACT
SSEC -.-&gt;|SSE stream| MARGIN

APIGW --&gt; AUTH
APIGW --&gt; DOC
APIGW --&gt; PDFM
APIGW --&gt; ENGINES
APIGW --&gt; MARGINMOD

AUTH --&gt; SUPAUTH
DOC --&gt; SUPDB
PDFM --&gt; STORAGE
PDFM --&gt; CHUNK
CHUNK --&gt; EMB
EMB --&gt; GEMAPI
EMB --&gt; SUPDB
VSEARCH --&gt; SUPDB

INT --&gt; AISVC
SOC --&gt; AISVC
CIT --&gt; AISVC
SCAF --&gt; AISVC
CIT --&gt; VSEARCH

AISVC --&gt; GEMADP
GEMADP --&gt; GEMAPI
AISVC --&gt; PROMPT
AISVC --&gt; BUDGET

MARGINMOD --&gt; MREPO
MARGINMOD --&gt; SSE_PUSH
MREPO --&gt; SUPDB
SSE_PUSH -.-&gt;|SSE| SSEC

ENGINES -.-&gt; TELEM
RAG -.-&gt; TELEM

%% Styling
classDef client fill:#EFF6FF,stroke:#226b90,stroke-width:1.5px,color:#181a1b
classDef gateway fill:#FEF3C7,stroke:#92400e,stroke-width:2px,color:#181a1b
classDef module fill:#f5f8fb,stroke:#354d59,color:#181a1b
classDef engine fill:#FFF7ED,stroke:#be6a4e,stroke-width:1.5px,color:#181a1b
classDef ext fill:#f1f5f9,stroke:#475569,stroke-width:1.5px,color:#181a1b

class REACT,MARGIN,IDLE,SSEC client
class APIGW gateway
class AUTH,DOC,PDFM,CHUNK,EMB,VSEARCH,MREPO,SSE_PUSH,AISVC,GEMADP,PROMPT,BUDGET,LOG,METRICS module
class INT,SOC,CIT,SCAF engine
class SUPAUTH,SUPDB,GEMAPI,STORAGE ext
```

A.3 Interceptor Sequence (Figure 6.1)

```
%% Sequence: Prompt Interceptor Flow
sequenceDiagram
    autonumber
    actor U as Writer
    participant FE as React + Tiptap
    participant GW as FastAPI Gateway
    participant INT as Interceptor Engine
    participant AI as AIService (Gemini)
    participant DB as Postgres
    participant MR as Margin Repository

    U-&gt;&gt;FE: Types prompt:&lt;br/&gt;"write a paragraph about X"
    FE-&gt;&gt;FE: Detect delegative pattern&lt;br/&gt;(regex + heuristics)
    FE-&gt;&gt;GW: POST /interceptor&lt;br/&gt;{paragraph_context, prompt_text}
    GW-&gt;&gt;GW: Validate JWT + rate limit
    GW-&gt;&gt;INT: classify_prompt()
    INT-&gt;&gt;AI: classify via Gemini&lt;br/&gt;(behavior tag classifier)
    AI--&gt;&gt;INT: {tag: "delegative",&lt;br/&gt;confidence: 0.92}
    INT-&gt;&gt;AI: generate_socratic_reframing&lt;br/&gt;(prompt template: [Spark Gap])
    AI--&gt;&gt;INT: "What claim must your&lt;br/&gt;reader accept first?"
    INT-&gt;&gt;MR: persist intervention&lt;br/&gt;(type=interceptor, status=active)
    MR-&gt;&gt;DB: INSERT intervention
    INT--&gt;&gt;GW: {action: "reframe",&lt;br/&gt;socratic_question, intervention_id}
    GW--&gt;&gt;FE: 200 OK + intervention payload
    FE-&gt;&gt;FE: Render ghost question in&lt;br/&gt;Living Margin (NOT in body)
```

```
    FE--&gt;&gt;U: Margin displays:&lt;br/&gt;"What claim must your&lt;br/&gt;reader accept first?"
    U-&gt;&gt;FE: Writer types their own answer&lt;br/&gt;(original prose preserved)
```

A.4 Citation Sentinel Sequence (Figure 6.2)

```
%% Sequence: Citation Sentinel RAG Flow
sequenceDiagram
    autonumber
    actor U as Writer
    participant FE as React + Tiptap
    participant GW as FastAPI Gateway
    participant CIT as Citation Sentinel
    participant VS as Vector Search
    participant DB as Postgres + pgvector
    participant AI as AIService (Gemini)
    participant MR as Margin Repository

    U-&gt;&gt;FE: Selects paragraph + clicks "Verify Claim"
    FE-&gt;&gt;GW: POST /citation-sentinel/verify&lt;br/&gt;{project_id, paragraph_id, claim_text}
    GW-&gt;&gt;GW: JWT + rate limit
    GW-&gt;&gt;CIT: verify_claim()
    CIT-&gt;&gt;CIT: Extract atomic claims&lt;br/&gt;from paragraph
    CIT-&gt;&gt;VS: retrieve_top_k(claim_embedding, k=5)
    VS-&gt;&gt;DB: SELECT chunks ORDER BY&lt;br/&gt;embedding &lt;=&gt; claim_embedding&lt;br/&gt;LIMIT 5
    DB--&gt;&gt;VS: top 5 PDF chunks&lt;br/&gt;with source metadata
    VS--&gt;&gt;CIT: retrieved_evidence[]
    CIT-&gt;&gt;AI: Verify claim against evidence&lt;br/&gt;(prompt template: [Lens: Sceptical Examiner])
    AI--&gt;&gt;CIT: {verdict: "partially_supported",&lt;br/&gt;supported_claims: [...],&lt;br/&gt;unsupported_claims: [...],&lt;br/&gt;citations: [chunk_ids]}
    CIT-&gt;&gt;MR: persist intervention&lt;br/&gt;(type=citation, verdict, citations)
    MR-&gt;&gt;DB: INSERT intervention +&lt;br/&gt;intervention_evidence junction
    CIT--&gt;&gt;GW: {verdict, citations,&lt;br/&gt;intervention_id}
    GW--&gt;&gt;FE: 200 OK + verdict payload
    FE-&gt;&gt;FE: Render verdict card in Living Margin&lt;br/&gt;with clickable chunk references
    FE--&gt;&gt;U: Margin shows:&lt;br/&gt;"Partially Supported" +&lt;br/&gt;3 source passages highlighted
    U-&gt;&gt;FE: Clicks source passage
    FE-&gt;&gt;GW: GET /pdfs/{pdf_id}/chunks/{chunk_id}
    GW--&gt;&gt;FE: chunk_text + page_number
    FE--&gt;&gt;U: Modal displays source passage
```

A.5 Socratic Spacer Sequence (Figure 6.3)

```
%% Sequence: Socratic Spacer Idle-Detection Flow
sequenceDiagram
    autonumber
    actor U as Writer
    participant FE as React + Tiptap
    participant ID as Idle Detector&lt;br/&gt;(frontend timer)
    participant GW as FastAPI Gateway
    participant SOC as Socratic Spacer Engine
    participant AI as AIService (Gemini)
    participant SSE as SSE Push Service
    participant DB as Postgres

    Note over U,FE: Writer stops typing for 45s
    U-&gt;&gt;FE: (inactivity - no keystrokes)
    FE-&gt;&gt;ID: keystroke timer expires&lt;br/&gt;(threshold: 45s)
    ID-&gt;&gt;FE: emit idle event with&lt;br/&gt;current paragraph context
    FE-&gt;&gt;GW: POST /socratic-spacer&lt;br/&gt;{project_id, paragraph_id,&lt;br/&gt;last_text, cursor_position}
    GW-&gt;&gt;GW: JWT + rate limit
    GW-&gt;&gt;SOC: generate_ghost_question()
    SOC-&gt;&gt;SOC: Build context window&lt;br/&gt;(last 3 paragraphs +&lt;br/&gt;current paragraph stub)
    SOC-&gt;&gt;AI: Generate Socratic question&lt;br/&gt;(prompt template: [Provocation])
    AI--&gt;&gt;SOC: "What evidence would&lt;br/&gt;make this claim false?"
    SOC-&gt;&gt;SSE: push intervention&lt;br/&gt;(type=socratic, intervention_id)
    SOC-&gt;&gt;DB: INSERT intervention&lt;br/&gt;(status=active)
    SSE--&gt;&gt;FE: SSE event:&lt;br/&gt;intervention payload
    FE-&gt;&gt;FE: Render ghost question&lt;br/&gt;in Living Margin (dimmed,&lt;br/&gt;non-blocking)
    FE--&gt;&gt;U: Margin shows ghost question:&lt;br/&gt;"What evidence would&lt;br/&gt;make this claim false?"

    alt Writer engages with question
        U-&gt;&gt;FE: Clicks "Explore this"
        FE-&gt;&gt;GW: PATCH /interventions/{id}&lt;br/&gt;{action: "engaged"}
        GW-&gt;&gt;DB: UPDATE intervention&lt;br/&gt;SET user_action='engaged'
        FE--&gt;&gt;U: Question promotes to&lt;br/&gt;primary intervention card
    else Writer dismisses
        U-&gt;&gt;FE: Clicks "Dismiss"
        FE-&gt;&gt;GW: PATCH /interventions/{id}&lt;br/&gt;{action: "dismissed"}
        GW-&gt;&gt;DB: UPDATE intervention&lt;br/&gt;SET user_action='dismissed'
        FE--&gt;&gt;U: Margin clears ghost question
    else Writer keeps typing (ignores)
        U-&gt;&gt;FE: Resumes typing
        FE-&gt;&gt;ID: Reset idle timer
```

```
        Note over FE: Ghost question auto-fades&lt;br/&gt;after 90s of inactivity
    end
```

A.6 Entity-Relationship Diagram (Figure 7.1)

```
%% Cogdoc ER Diagram
erDiagram
    USER ||--o{ PROJECT : owns
    USER ||--o{ INTERVENTION : triggers
    USER ||--|| AI_PROVIDER_CONFIG : configures
    PROJECT ||--|| DOCUMENT : contains
    PROJECT ||--o{ PDF : has
    DOCUMENT ||--o{ PARAGRAPH : contains
    PARAGRAPH ||--o{ INTERVENTION : receives
    PDF ||--o{ PDF_CHUNK : parsed_into
    PDF_CHUNK ||--|| EMBEDDING : has
    PDF_CHUNK ||--o{ INTERVENTION_EVIDENCE : cited_in
    INTERVENTION ||--o{ INTERVENTION_EVIDENCE : references
    PROMPT_TEMPLATE ||--o{ INTERVENTION : generates

    USER {
        uuid id PK
        string email UK
        string display_name
        timestamp created_at
        timestamp last_login
        int daily_token_budget
    }

    PROJECT {
        uuid id PK
        uuid user_id FK
        string title
        text description
        timestamp created_at
        timestamp updated_at
        boolean archived
    }

    DOCUMENT {
        uuid id PK
        uuid project_id FK
        text content
        jsonb tiptap_state
        timestamp last_saved
    }

    PARAGRAPH {
        uuid id PK
        uuid document_id FK
        int order_index
        text content
        timestamp created_at
        timestamp updated_at
    }

    PDF {
        uuid id PK
        uuid project_id FK
        string filename
        string storage_path
        bigint size_bytes
        string mime_type
        int page_count
        timestamp uploaded_at
        string parse_status
    }

    PDF_CHUNK {
        uuid id PK
        uuid pdf_id FK
        int chunk_index
        text content
        int page_number
        int char_count
        timestamp created_at
    }

    EMBEDDING {
        uuid id PK
        uuid chunk_id FK
        vector embedding
        string model_version
        timestamp created_at
```

```
    }

    INTERVENTION {
        uuid id PK
        uuid user_id FK
        uuid project_id FK
        uuid paragraph_id FK
        uuid prompt_template_id FK
        string engine_type
        string behavior_tag
        text prompt_text
        text response_text
        string verdict
        string status
        string user_action
        timestamp created_at
        timestamp resolved_at
    }

    INTERVENTION_EVIDENCE {
        uuid id PK
        uuid intervention_id FK
        uuid chunk_id FK
        float similarity_score
        text cited_excerpt
    }

    PROMPT_TEMPLATE {
        uuid id PK
        string behavior_tag
        string name
        text template
        string model_id
        int version
        boolean active
        timestamp created_at
    }

    AI_PROVIDER_CONFIG {
        uuid id PK
        uuid user_id FK
        string provider
        string api_key_encrypted
        jsonb model_preferences
        timestamp updated_at
    }
```

A.7 AI Service Class Diagram (Figure 9.1)

```
%% Cogdoc AI Service Layer - Class Diagram
classDiagram
    direction TB

    %% Interface
    class AIService {
        &lt;&lt;interface&gt;&gt;
        +classify_prompt(text: str, context: dict) ClassifyResult
        +generate_socratic(question_type: BehaviorTag, context: dict) SocraticResponse
        +verify_claim(claim: str, evidence: List~Chunk~) VerdictResult
        +generate_embedding(text: str) Vector
        +generate_scaffold(outline_type: str, context: dict) ScaffoldResult
    }

    %% Abstract base
    class BaseAdapter {
        &lt;&lt;abstract&gt;&gt;
        #provider: str
        #api_key: str
        #model_map: dict
        #token_budget: TokenBudget
        +call_api(method, payload) Response
        #track_tokens(used: int)
        #fallback_to_default_model()
    }

    %% Concrete adapters
    class GeminiAdapter {
        -gemini_client: GeminiSDK
        +classify_prompt(text, context) ClassifyResult
        +generate_socratic(question_type, context) SocraticResponse
        +verify_claim(claim, evidence) VerdictResult
        +generate_embedding(text) Vector
        +generate_scaffold(outline_type, context) ScaffoldResult
        -build_classification_prompt(text) str
```

```
    -build_verification_prompt(claim, evidence) str
}

class OpenAIAdapter {
    -openai_client: OpenAISDK
    +classify_prompt(text, context) ClassifyResult
    +generate_socratic(question_type, context) SocraticResponse
    +verify_claim(claim, evidence) VerdictResult
    +generate_embedding(text) Vector
}

class LocalLLMAdapter {
    -model_path: str
    +classify_prompt(text, context) ClassifyResult
    +generate_socratic(question_type, context) SocraticResponse
    +verify_claim(claim, evidence) VerdictResult
}

%% Supporting classes
class PromptTemplateRegistry {
    -templates: dict
    +get(behavior_tag: str) PromptTemplate
    +register(template: PromptTemplate)
    +activate(version: int)
    +deactivate(version: int)
}

class PromptTemplate {
    +id: uuid
    +behavior_tag: BehaviorTag
    +name: str
    +template: str
    +model_id: str
    +version: int
    +active: bool
    +render(context: dict) str
}

class TokenBudget {
    +daily_limit: int
    +used_today: int
    +reset_at: timestamp
    +check_available(amount: int) bool
    +consume(amount: int)
    +reset()
}

class BehaviorTag {
    &lt;&lt;enumeration&gt;&gt;
    PROVOCATION
    LENS_PERSPECTIVE
    STRUCTURAL_SCAFFOLD
    THE_SPARK_GAP
}

class ClassifyResult {
    +tag: str
    +confidence: float
    +is_delegative: bool
}

class VerdictResult {
    +verdict: str
    +supported_claims: list
    +unsupported_claims: list
    +citations: list
}

%% Relationships
AIService &lt;|.. BaseAdapter : implements
BaseAdapter &lt;|-- GeminiAdapter
BaseAdapter &lt;|-- OpenAIAdapter
BaseAdapter &lt;|-- LocalLLMAdapter
BaseAdapter o-- PromptTemplateRegistry : uses
BaseAdapter o-- TokenBudget : tracks
PromptTemplateRegistry o-- PromptTemplate : manages
PromptTemplate --&gt; BehaviorTag : tagged with
GeminiAdapter ..&gt; ClassifyResult : produces
GeminiAdapter ..&gt; VerdictResult : produces
```